Лабораторная работа № 3

Анализ тензорных данных с использованием свёрточных нейронных сетей

Цель работы – сформировать практические навыки анализа тензорных данных с использованием свёрточных нейронных сетей, а также развить понимание математических основ свёртки, корреляции, размерности выходных тензоров, числа параметров и рецептивного поля.

Задачи работы

1. Изучить представление данных в виде тензоров и особенности свёрточных операций.
2. Выполнить аналитический расчёт:
   – размеров выходных тензоров;
   – числа обучаемых параметров;
   – рецептивного поля свёрточной сети.
3. Реализовать базовую операцию двумерной кросс-корреляции вручную.
4. Сравнить ручную реализацию с torch.nn.functional.conv2d.
5. Построить и обучить свёрточную нейронную сеть для анализа тензорных данных.
6. Провести визуальный анализ фильтров и карт признаков.
7. Сделать выводы о применимости свёрточных архитектур к данным с пространственной или локальной структурой.

1. Теоретическая справка

1.1. Тензорное представление данных

В глубоком обучении данные часто представляются в виде тензоров. Для изображений и схожих с ними данных обычно используется четырёхмерный тензор:

$$X \in \mathbb{R}^{N \times C \times H \times W},$$

где N — размер мини-пакета;
C — число каналов;
H — высота;
W — ширина.

Например:

– чёрно-белые изображения имеют $C = 1$;
– цветные изображения обычно имеют $C = 3$;
– многоканальные сигналы, спектрограммы или физические поля также могут быть представлены как тензоры с одним или несколькими каналами.

> **Замечание.**
> Свёрточные сети особенно эффективны, когда данные обладают локальной структурой: соседние элементы тензора статистически связаны, а полезные признаки могут появляться в разных участках входного тензора.

1.2. Свёртка и кросс-корреляция

В математическом смысле свёртка включает отражение ядра. В большинстве библиотек глубокого обучения, включая PyTorch и TensorFlow, под «свёрткой» обычно реализуется операция кросс-корреляции.

Для одноканального входа и одного фильтра операция кросс-корреляции может быть записана как:

$$y_{i,j} = \sum_{p=0}^{k_h-1} \sum_{q=0}^{k_w-1} x_{is_h+p,\, js_w+q}\; w_{p,q} + b,$$

где x — входное поле;

y — выходное поле;
w — фильтр ядра;
b — смещение;
k_h, k_w — высота и ширина ядра;
s_h, s_w — шаги свёртки по высоте и ширине;
i, j — индексы выходных позиций.

Для многоканального входа и нескольких выходных каналов:

$$y_{c_{\text{out}},i,j} = \sum_{c_{\text{in}}=0}^{C_{\text{in}}-1} \sum_{p=0}^{k_h-1} \sum_{q=0}^{k_w-1} x_{c_{\text{in}},is_h+p,\, js_w+q}\; W_{c_{\text{out}},c_{\text{in}},p,q} + b_{c_{\text{out}}}.$$

где C_{in} — число входных каналов;
c_{out} — индекс выходного канала;
c_{in} — индекс входного канала;
W — четырёхмерный тензор весов свёрточного слоя;
$b_{c_{\text{out}}}$ — смещение для выходного канала.

> **Важно!**
> В `torch.nn.Conv2d` и `tf.keras.layers.Conv2D` реализуется именно кросс-корреляция, а не математическая свёртка с отражением ядра. Для обучения нейронных сетей это непринципиально, так как значения ядра подбираются в процессе оптимизации.

1.3. Размер выходного тензора

Пусть вход имеет размер $H_{\text{in}} \times W_{\text{in}}$, используются параметры:

- k_h, k_w — размер ядра;
- s_h, s_w — шаг;
- p_h, p_w — заполнение;
- d_h, d_w — дилатация.

Тогда размер выхода по высоте:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p_h - d_h(k_h - 1) - 1}{s_h} + 1 \right\rfloor.$$

Аналогично по ширине:

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2p_w - d_w(k_w - 1) - 1}{s_w} + 1 \right\rfloor.$$

Частный случай без дилатации, с $k = 3$, $s = 1$, $p = 1$:

$$H_{\text{out}} = H_{\text{in}}, \quad W_{\text{out}} = W_{\text{in}}.$$

Такой режим часто называют same convolution, поскольку пространственные размеры сохраняются.

1.4. Число параметров свёрточного слоя

Для свёрточного слоя с параметрами:

- C_{in} — входные каналы;
- C_{out} — выходные каналы;
- k_h, k_w — размеры ядра;

число обучаемых весов:

$$P_w = C_{\text{out}} \cdot C_{\text{in}} \cdot k_h \cdot k_w.$$

Если используется смещение, добавляется ещё:

$$P_b = C_{\text{out}}.$$

Итого:

$$P = C_{\text{out}} \cdot C_{\text{in}} \cdot k_h \cdot k_w + C_{\text{out}}.$$

> **Упражнение.**
> Рассчитайте число параметров слоя `Conv2d(3, 32, kernel_size=3)`.
> Решение: $32 \cdot 3 \cdot 3 \cdot 3 + 32 = 896$.

1.5. Рецептивное поле

Рецептивное поле — **это область входного тензора, влияющая на конкретный элемент выходного тензора**.

Для последовательности свёрточных слоёв можно использовать рекуррентные соотношения:

$$r_l = r_{l-1} + (k_l - 1)\, J_{l-1},$$

$$J_l = J_{l-1} \cdot s_l,$$

где r_l — рецептивное поле после слоя l;
J_l — шаг рецептивного поля относительно входа;
k_l — размер ядра на слое l;
s_l — шаг свёртки на слое l;
начальные условия: $r_0 = 1, J_0 = 1$.

> **Замечание.**
> Padding напрямую не увеличивает рецептивное поле в классической рекуррентной формуле, но влияет на доступность краевых пикселей и размер карты признаков.

1.6. Почему свёрточные сети подходят для тензорных данных

Свёрточные архитектуры используют два важных индуктивных предположения:

1. **Локальная связность.** Каждый нейрон обрабатывает только локальную область входа.
2. **Разделение весов.** Один и тот же фильтр применяется ко всем пространственным позициям.

Это даёт:

- уменьшение числа параметров по сравнению с полносвязными слоями;
- устойчивость к малым сдвигам признаков;
- иерархическое представление признаков: от простых к более сложным;
- естественную работу с данными, имеющими регулярную сеточную структуру.

1.7. Типичные ошибки студентов

1. **Путают свёртку и кросс-корреляцию.** В библиотеках глубокого обучения обычно используется кросс-корреляция.
2. **Неправильно рассчитывают размеры выхода.** Особенно при использовании stride > 1, padding = 0 или dilation > 1.
3. **Игнорируют число каналов.** Например, подают тензор формы (N, H, W) вместо (N, C, H, W).
4. **Считают параметры только по последнему слою.** В CNN значительная часть параметров может находиться как в свёрточных, так и в полносвязных слоях.
5. **Не учитывают рецептивное поле.** В результате сеть может быть слишком мелкой для выделения крупных структур.

2. Требования к программному обеспечению и исходным данным

Рекомендуемое программное обеспечение

- Python 3.10 или новее;
- PyTorch;
- NumPy;
- Matplotlib;
- при необходимости — TensorFlow/Keras для альтернативной реализации.

Данные

Для выполнения работы могут использоваться:

- MNIST;
- FashionMNIST;
- KMNIST; EMNIST;
- CIFAR-10;
- CIFAR-100;
- USPS;
- SVHN;
- синтетические тензорные данные.

Выбор конкретного набора определяется индивидуальным вариантом.

3. Пошаговое задание с контрольными точками

Шаг 0. Фиксация случайности и подготовка окружения

```
import os
import random

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset, random_split
from torchvision import datasets, transforms
import matplotlib.pyplot as plt

def set_seed(seed: int = 42) -> None:
    """
    Фиксирует случайные генераторы для воспроизводимости эксперимента.
    """
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
set_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Используемое устройство:", device)
```

Шаг 1. Аналитический расчёт свёрточных слоёв

Перед построением сети необходимо уметь вручную рассчитывать размеры выхода и число параметров.

Расчёт длины выходной оси

```
def conv_output_length(
    input_length: int,
    kernel_size: int,
    stride: int = 1,
    padding: int = 0,
    dilation: int = 1
) -> int:
```

```
    """
    Возвращает размер выходного тензора по одной оси.
    """
    numerator = input_length + 2 * padding - dilation * (kernel_size - 1) - 1
    return numerator // stride + 1
```

Расчёт числа параметров свёрточного слоя

```
def conv2d_num_params(
    in_channels: int,
    out_channels: int,
    kernel_size: int,
    use_bias: bool = True
) -> int:
    """
    Возвращает число параметров свёрточного слоя.
    """
    params = out_channels * in_channels * kernel_size * kernel_size
    if use_bias:
        params += out_channels
    return params
```

Пример расчёта

```
H_in, W_in = 28, 28
C_in = 1
H1 = conv_output_length(H_in, kernel_size=3, stride=1, padding=1)
W1 = conv_output_length(W_in, kernel_size=3, stride=1, padding=1)
print("Размер после первого свёрточного слоя:", (H1, W1))
params1 = conv2d_num_params(
    in_channels=C_in,
    out_channels=16,
    kernel_size=3,
    use_bias=True
)
print("Число параметров первого слоя:", params1)
```

Контрольная точка 1

Студент должен представить:

1. Таблицу размеров тензора после каждого слоя для архитектуры из индивидуального варианта.
2. Расчёт числа параметров для каждого свёрточного слоя.
3. Оценку рецептивного поля после каждого свёрточного блока.
4. Сравнение ручного расчёта с фактическими размерами, выводимыми моделью.

Шаг 2. Ручная реализация двумерной кросс-корреляции

Цель этапа — понять, как именно свёрточный слой обрабатывает входной тензор.

Ниже приведена реализация через `torch.nn.functional.unfold`, которая эффективно раскладывает локальные окна в столбцы.

```
def manual_conv2d(
    x: torch.Tensor,
    weight: torch.Tensor,
    bias: torch.Tensor = None,
    stride: int = 1,
    padding: int = 0
```

```
) -> torch.Tensor:
    """
    Учебная реализация двумерной кросс-корреляции.
    Аргументы:
        x: входной тензор формы (N, C_in, H, W);
        weight: веса формы (C_out, C_in, kH, kW);
        bias: смещение формы (C_out,) или None;
        stride: шаг свёртки;
        padding: заполнение.
    Возвращает:
        тензор формы (N, C_out, H_out, W_out).
    """
    if isinstance(stride, int):
        stride = (stride, stride)

    if isinstance(padding, int):
        padding = (padding, padding)
    N, C_in, H, W = x.shape
    C_out, C_in_w, kH, kW = weight.shape

    assert C_in == C_in_w, "Число входных каналов не совпадает"

    # Заполнение границ
    x_pad = F.pad(x, (padding[1], padding[1], padding[0], padding[0]))
    H_pad, W_pad = x_pad.shape[2], x_pad.shape[3]
    # Размеры выхода без дилатации
    H_out = (H_pad - kH) // stride[0] + 1
    W_out = (W_pad - kW) // stride[1] + 1
    # Раскладка локальных окон в столбцы
    # Результат: (N, C_in * kH * kW, H_out * W_out)
    col = F.unfold(x_pad, kernel_size=(kH, kW), stride=stride)
    # Приведение к виду (N * H_out * W_out, C_in * kH * kW)
    col = col.transpose(1, 2).reshape(N * H_out * W_out, -1)
    # Преобразование весов к виду (C_in * kH * kW, C_out)
    weight_col = weight.view(C_out, -1).t()
    # Матричное умножение
    out = col @ weight_col
    # Добавление смещения
    if bias is not None:
        out = out + bias

    # Возврат к тензорной форме
    out = out.view(N, H_out, W_out, C_out)
    out = out.permute(0, 3, 1, 2).contiguous()

    return out
```

Проверка корректности реализации

```
x_test = torch.randn(2, 1, 8, 8)
w_test = torch.randn(4, 1, 3, 3)
b_test = torch.randn(4)

out_manual = manual_conv2d(
    x_test,
    w_test,
```

```
    b_test,
    stride=1,
    padding=1
)

out_torch = F.conv2d(
    x_test,
    w_test,
    b_test,
    stride=1,
    padding=1
)
max_abs_error = torch.max(torch.abs(out_manual - out_torch)).item()
print("Максимальная абсолютная ошибка:", max_abs_error)
print("Совпадение:", torch.allclose(out_manual, out_torch, atol=1e-5))
```

Контрольная точка 2

Студент должен представить:

1. Код ручной реализации кросс-корреляции.
2. Результат сравнения с `F.conv2d`.
3. Максимальную абсолютную ошибку.
4. Вывод о корректности реализации.

Шаг 3. Построение свёрточной нейронной сети

Ниже приведена базовая настраиваемая архитектура, подходящая для многих вариантов.

```
class ConvBlock(nn.Module):
    """
    Свёрточный блок:
    Conv2d -> BatchNorm -> ReLU
    """
    def __init__(
        self,
        in_channels: int,
        out_channels: int,
        kernel_size: int = 3,
        stride: int = 1,
        padding: int = 1,
        use_bn: bool = True
    ):
        super().__init__()
        self.conv = nn.Conv2d(
            in_channels=in_channels,
            out_channels=out_channels,
            kernel_size=kernel_size,
            stride=stride,
            padding=padding
        )
        self.bn = nn.BatchNorm2d(out_channels) if use_bn else nn.Identity()
        self.act = nn.ReLU(inplace=True)
    def forward(self, x):
        return self.act(self.bn(self.conv(x)))
```

```python
class SimpleCNN(nn.Module):
    """
    Простая свёрточная сеть для анализа тензорных данных.
    """
    def __init__(
        self,
        in_channels: int = 1,
        num_classes: int = 10,
        base_channels: int = 16,
        use_bn: bool = True
    ):
        super().__init__()
        self.features = nn.Sequential(
            ConvBlock(in_channels, base_channels, 3, 1, 1, use_bn),
            nn.MaxPool2d(2),

            ConvBlock(base_channels, base_channels * 2, 3, 1, 1, use_bn),
            nn.MaxPool2d(2),

            ConvBlock(base_channels * 2, base_channels * 4, 3, 1, 1, use_bn),
            nn.AdaptiveAvgPool2d(1)
        )
        self.classifier = nn.Linear(base_channels * 4, num_classes)
    def forward(self, x):
        x = self.features(x)
        x = torch.flatten(x, start_dim=1)
        x = self.classifier(x)
        return x
```

Подсчёт числа параметров модели

```python
def count_parameters(model: nn.Module) -> int:
    """
    Возвращает число обучаемых параметров модели.
    """
    return sum(p.numel() for p in model.parameters() if p.requires_grad)


model = SimpleCNN(
    in_channels=1,
    num_classes=10,
    base_channels=16,
    use_bn=True
).to(device)

print("Число параметров модели:", count_parameters(model))
print(model)
```

Контрольная точка 3

Студент должен представить:

1. Архитектуру модели для своего варианта.
2. Общее число параметров.
3. Число параметров по слоям.
4. Фактические размеры тензоров после каждого блока.
5. Сравнение с ручным расчётом из контрольной точки 1.

Шаг 4. Обучение и оценка модели

Далее необходимо обучить модель на выбранном наборе данных.

Пример загрузки данных для MNIST

```
transform_grayscale = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform_grayscale
)

test_dataset = datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform_grayscale
)

train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=256, shuffle=False)
```

Функция обучения одной эпохи

```
def train_one_epoch(model, loader, optimizer, criterion, device):
    """
    Обучение модели в течение одной эпохи.
    """
    model.train()

    total_loss = 0.0
    total_correct = 0
    total_samples = 0

    for x_batch, y_batch in loader:
        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()

        logits = model(x_batch)
        loss = criterion(logits, y_batch)

        loss.backward()
        optimizer.step()

        total_loss += loss.item() * x_batch.size(0)
        total_correct += (logits.argmax(dim=1) == y_batch).sum().item()
        total_samples += x_batch.size(0)
```

```
    avg_loss = total_loss / total_samples
    accuracy = total_correct / total_samples

    return avg_loss, accuracy
```

Функция оценки

```
@torch.no_grad()
def evaluate(model, loader, criterion, device):
    """
    Оценка качества модели на заданном датасете.
    """
    model.eval()
    total_loss = 0.0
    total_correct = 0
    total_samples = 0
    for x_batch, y_batch in loader:
        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)
        logits = model(x_batch)
        loss = criterion(logits, y_batch)
        total_loss += loss.item() * x_batch.size(0)
        total_correct += (logits.argmax(dim=1) == y_batch).sum().item()
        total_samples += x_batch.size(0)

    avg_loss = total_loss / total_samples
    accuracy = total_correct / total_samples

    return avg_loss, accuracy
```

Полный цикл обучения

```
def run_cnn_experiment(
    model_class,
    train_loader,
    test_loader,
    in_channels: int,
    num_classes: int,
    lr: float = 1e-3,
    epochs: int = 5,
    device=device
):
    """
    Полный цикл обучения и оценки свёрточной сети.
    """
    model = model_class(
        in_channels=in_channels,
        num_classes=num_classes,
        base_channels=16,
        use_bn=True
    ).to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)

    history = {
        "train_loss": [],
```

```
        "train_acc": [],
        "test_loss": [],
        "test_acc": []
    }
    for epoch in range(1, epochs + 1):
        train_loss, train_acc = train_one_epoch(
            model,
            train_loader,
            optimizer,
            criterion,
            device
        )
        test_loss, test_acc = evaluate(
            model,
            test_loader,
            criterion,
            device
        )
        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["test_loss"].append(test_loss)
        history["test_acc"].append(test_acc)
        print(
            f"Эпоха {epoch:02d}: "
            f"train_loss={train_loss:.4f}, train_acc={train_acc:.4f}, "
            f"test_loss={test_loss:.4f}, test_acc={test_acc:.4f}"
        )
    return model, history
```

Запуск обучения

```
model, history = run_cnn_experiment(
    model_class=SimpleCNN,
    train_loader=train_loader,
    test_loader=test_loader,
    in_channels=1,
    num_classes=10,
    lr=1e-3,
    epochs=5,
    device=device
)
```

Построение графиков обучения

```
def plot_history(history):
    """
    Построение графиков обучения.
    """
    plt.figure(figsize=(8, 5))
    plt.plot(history["train_loss"], label="train_loss")
    plt.plot(history["test_loss"], label="test_loss")
    plt.xlabel("Эпоха")
    plt.ylabel("Потеря")
    plt.title("Функция потерь")
    plt.grid(True)
    plt.legend()
    plt.show()
```

```
    plt.figure(figsize=(8, 5))
    plt.plot(history["train_acc"], label="train_acc")
    plt.plot(history["test_acc"], label="test_acc")
    plt.xlabel("Эпоха")
    plt.ylabel("Точность")
    plt.title("Точность")
    plt.grid(True)
    plt.legend()
    plt.show()
plot_history(history)
```

Контрольная точка 4

Студент должен представить:

1. Графики обучающей и тестовой функции потерь.
2. Графики точности.
3. Итоговую таблицу:

– набор данных;
– архитектура;
– число параметров;
– точность на обучении;
– точность на тесте;
– потери на обучении;
– потери на тесте.

4. Вывод о наличии или отсутствии переобучения.

Шаг 5. Анализ фильтров и карт признаков

Свёрточные сети обучают фильтры, которые реагируют на локальные структуры. Визуализация этих фильтров помогает понять, какие признаки извлекает сеть.

Визуализация фильтров первого свёрточного слоя

```
def plot_first_conv_filters(model, max_filters: int = 16):
    """
    Визуализирует фильтры первого свёрточного слоя.
    Для многоканального входа усредняет фильтры по входным каналам.
    """
    conv = None
    for module in model.modules():
        if isinstance(module, nn.Conv2d):
            conv = module
            break
    if conv is None:
        print("Свёрточный слой не найден")
        return
    weights = conv.weight.detach().cpu()
    n_filters = min(weights.shape[0], max_filters)
    cols = 4
    rows = (n_filters + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(8, 2 * rows))
    axes = np.atleast_1d(axes)
    for i in range(n_filters):
        kernel = weights[i].mean(dim=0)
        axes[i].imshow(kernel.numpy(), cmap="gray")
```

```
        axes[i].axis("off")
    for j in range(n_filters, len(axes)):
        axes[j].axis("off")
    plt.suptitle("Фильтры первого свёрточного слоя")
    plt.show()
plot_first_conv_filters(model)
```

Получение карт признаков

```
def get_conv_feature_maps(model, x):
    """
    Возвращает карты признаков для всех свёрточных слоёв.
    """
    feature_maps = []
    hooks = []
    def hook(module, input, output):
        feature_maps.append(output.detach())
    for module in model.modules():
        if isinstance(module, nn.Conv2d):
            hooks.append(module.register_forward_hook(hook))
    model.eval()
    with torch.no_grad():
        model(x.to(device))
    for h in hooks:
        h.remove()
    return feature_maps
```

Визуализация карт признаков

```
def plot_feature_maps(feature_maps, layer_idx: int = 0, max_maps: int = 8):
    """
    Визуализирует карты признаков выбранного свёрточного слоя.
    """
    fm = feature_maps[layer_idx][0].cpu()
    n_maps = min(fm.shape[0], max_maps)
    cols = 4
    rows = (n_maps + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(8, 2 * rows))
    axes = np.atleast_1d(axes)
    for i in range(n_maps):
        axes[i].imshow(fm[i].numpy(), cmap="viridis")
        axes[i].axis("off")
    for j in range(n_maps, len(axes)):
        axes[j].axis("off")
    plt.suptitle(f"Карты признаков слоя {layer_idx + 1}")
    plt.show()
sample_batch, _ = next(iter(test_loader))
feature_maps = get_conv_feature_maps(model, sample_batch[:1])
plot_feature_maps(feature_maps, layer_idx=0, max_maps=8)
```

Контрольная точка 5

Студент должен представить:

1. Визуализацию фильтров первого свёрточного слоя.
2. Визуализацию карт признаков для одного или нескольких примеров.
3. Письменную интерпретацию:

– какие структуры обнаруживают фильтры;

– как меняется представление признаков по глубине сети;
– согласуются ли визуальные наблюдения с задачей классификации.

6. Альтернативный фрагмент на TensorFlow / Keras

Ниже приведён альтернативный способ реализации и использования свёрточных операций в TensorFlow.

Ручная кросс-корреляция в TensorFlow

```
import tensorflow as tf
def manual_conv2d_tf(
    x: tf.Tensor,
    filters: tf.Tensor,
    bias: tf.Tensor = None,
    stride: int = 1,
    padding: int = 0
) -> tf.Tensor:
    """
    Учебная реализация кросс-корреляции в TensorFlow.
    Аргументы:
        x: тензор формы (N, H, W, C_in);
        filters: фильтры формы (kH, kW, C_in, C_out);
        bias: смещение формы (C_out,) или None;
        stride: шаг свёртки;
        padding: заполнение.
    Возвращает:
        тензор формы (N, H_out, W_out, C_out).
    """
    if padding > 0:
        x = tf.pad(
            x,
            [[0, 0], [padding, padding], [padding, padding], [0, 0]],
            mode="CONSTANT"
        )
    # Извлечение локальных окон
    patches = tf.image.extract_patches(
        images=x,
        sizes=[1, filters.shape[0], filters.shape[1], 1],
        strides=[1, stride, stride, 1],
        rates=[1, 1, 1, 1],
        padding="VALID"
    )
    # Преобразование фильтров к матричной форме
    filters_flat = tf.reshape(filters, [-1, filters.shape[-1]])
    # Матричное умножение
    out = tf.matmul(patches, filters_flat)
    if bias is not None:
        out = out + bias
    return out
```

Пример использования в Keras

```
def make_keras_cnn(input_shape=(28, 28, 1), num_classes=10):
    """
    Простая свёрточная сеть в Keras.
    """
    model = tf.keras.Sequential([
```

```
        tf.keras.layers.Input(shape=input_shape),
        tf.keras.layers.Conv2D(16, 3, padding="same", activation="relu"),
        tf.keras.layers.MaxPooling2D(),

        tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
        tf.keras.layers.MaxPooling2D(),

        tf.keras.layers.Conv2D(64, 3, padding="same", activation="relu"),
        tf.keras.layers.GlobalAveragePooling2D(),

        tf.keras.layers.Dense(num_classes, activation="softmax")
    ])
    model.compile(
        optimizer=tf.keras.optimizers.AdamW(learning_rate=1e-3),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model
```

> **Замечание.**
> В TensorFlow/Keras по умолчанию используется формат данных `channels_last`: (N, H, W, C), тогда как в PyTorch обычно используется `channels_first`: (N, C, H, W). Это необходимо учитывать при переносе данных и расчётах.

4. Критерии оценивания

Критерий	Баллы	Описание
Аналитический расчёт размеров и параметров	2	Верно рассчитаны размеры выхода, параметры и рецептивное поле
Ручная реализация свёртки	2	Реализация корректна, ошибка сравнения с библиотекой мала
Построение и обучение CNN	3	Модель обучена, метрики представлены, переобучение проанализировано
Анализ фильтров и карт признаков	2	Визуализации выполнены, дана содержательная интерпретация
Защита работы	1	Студент отвечает на вопросы, понимает математическую структуру CNN
Итого	**10**	Максимальный балл за работу — 10

Шкала перевода

- **9–10 баллов** — отлично;
- **7–8 баллов** — хорошо;
- **5–6 баллов** — удовлетворительно;
- **менее 5 баллов** — неудовлетворительно.

5. Вопросы для отчёта

1. Чем математическая свёртка отличается от кросс-корреляции?
2. Почему в библиотеках глубокого обучения обычно используется кросс-корреляция?
3. Как параметры `stride`, `padding` и `dilation` влияют на размер выходного тензора?
4. Как рассчитать число параметров свёрточного слоя?

5. Почему свёрточные слои обычно имеют меньше параметров, чем полносвязные при тех же пространственных размерах входа?
6. Что такое рецептивное поле и как оно изменяется с глубиной сети?
7. Какие индуктивные предположения заложены в свёрточные сети?
8. Почему для цветных изображений вход имеет три канала?
9. Как изменяется число каналов и пространственные размеры при использовании `MaxPool2d`?
10. Какие признаки обычно извлекают первые свёрточные слои?
11. Почему глобальный средний пулинг часто предпочтительнее больших полносвязных слоёв?
12. Какие признаки переобучения можно обнаружить по графикам обучающей и тестовой ошибки?

6. Варианты индивидуальных заданий

Ниже приведены 10 вариантов индивидуальных заданий. Каждый вариант задаёт:

1) набор данных;
2) тип входного тензора;
3) архитектурные особенности;
4) дополнительное аналитическое задание.

Таблица вариантов

№	Набор данных	Входной тензор	Архитектурные особенности	Дополнительное задание
1	MNIST	$1 \times 28 \times 28$	3 свёрточных блока, `MaxPool2d`	Рассчитать рецептивное поле
2	FashionMNIST	$1 \times 28 \times 28$	3 свёрточных блока с `BatchNorm2d`	Сравнить число параметров по слоям
3	KMNIST	$1 \times 28 \times 28$	LeNet-подобная архитектура с ядрами 5×5	Сравнить 3×3 и 5×5 фильтры
4	EMNIST (digits)	$1 \times 28 \times 28$	2 свёрточных блока и глобальный пулинг	Визуализировать карты признаков
5	CIFAR-10, подвыборка 5 классов	$3 \times 32 \times 32$	4 свёрточных блока	Рассчитать параметры цветного входа
6	CIFAR-100, подвыборка 10 классов	$3 \times 32 \times 32$	4 свёрточных блока с `Dropout2d`	Проанализировать переобучение
7	USPS	$1 \times 16 \times 16$	Адаптивная архитектура под малый вход	Проверить корректность размеров после пулинга
8	Синтетические паттерны	$1 \times 32 \times 32$	Классификация горизонтальных, вертикальных и диагональных структур	Показать ориентационную избирательность фильтров

№	Набор данных	Входной тензор	Архитектурные особенности	Дополнительное задание
9	Синтетические многоканальные паттерны	$3 \times 32 \times 32$	Классификация комбинаций каналов	Проанализировать смешение каналов в первом слое
10	CIFAR-10, полная версия	$3 \times 32 \times 32$	4 свёрточных блока и расширенная часть с числом каналов 32–64–128	Сравнить ручную свёртку и Conv2D на примере первого слоя

6.1. Генератор синтетических тензорных данных

Для вариантов 8 и 9 можно использовать следующий код.

```python
def generate_synthetic_tensors(
    n_samples: int = 3000,
    size: int = 32,
    channels: int = 1,
    num_classes: int = 3,
    noise_std: float = 0.1
) -> TensorDataset:
    """
    Генерирует синтетические тензорные данные:
    класс 0 — горизонтальные полосы;
    класс 1 — вертикальные полосы;
    класс 2 — диагональные структуры.
    """
    x = torch.zeros(n_samples, channels, size, size, dtype=torch.float32)
    y = torch.randint(0, num_classes, (n_samples,), dtype=torch.long)

    coords = torch.arange(size)

    for idx, label in enumerate(y):
        img = torch.zeros(channels, size, size, dtype=torch.float32)

        if label == 0:
            # Горизонтальные полосы
            img[..., :, ::4] = 1.0

        elif label == 1:
            # Вертикальные полосы
            img[..., ::4, :] = 1.0

        else:
            # Диагональные структуры
            mask = ((coords[None, :] + coords[:, None]) % 8 < 2).float()
            img[:] = mask.unsqueeze(0)

        # Добавляем шум
        img = img + torch.randn_like(img) * noise_std
        img = torch.clamp(img, 0.0, 1.0)

        x[idx] = img

    return TensorDataset(x, y)
```

6.2. Пример использования синтетического датасета

```
synthetic_dataset = generate_synthetic_tensors(
    n_samples=3000,
    size=32,
    channels=1,
    num_classes=3
)

train_size = int(0.8 * len(synthetic_dataset))
val_size = len(synthetic_dataset) - train_size

synthetic_train, synthetic_val = random_split(
    synthetic_dataset,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(42)
)

synthetic_train_loader = DataLoader(synthetic_train, batch_size=64, shuffle=Tru
e)
synthetic_val_loader = DataLoader(synthetic_val, batch_size=128, shuffle=False)
```

7. Требования к содержанию отчёта

Отчёт должен включать:

1. Титульную информацию.
2. Цель и задачи работы.
3. Краткую теоретическую справку.
4. Аналитический расчёт:
   – размеры выходных тензоров;
   – число параметров;
   – рецептивное поле.
5. Код ручной реализации кросс-корреляции.
6. Результат сравнения с библиотечной реализацией.
7. Архитектуру использованной модели.
8. Графики обучения.
9. Таблицу итоговых метрик.
10. Визуализацию фильтров и карт признаков.
11. Письменные выводы.
12. Ответы на контрольные вопросы.

Литература

1. Ле Кун Я., Ботту Л., Бенджио И., Хаффнер П. Градиентное обучение применительно к свёрточным сетям // Труды Института инженеров по электротехнике и электронике. — 1998. — Т. 86, № 11. — С. 2278–2324.
2. Крижевский А., Суцкевер И., Хинтон Г. Классификация изображений с помощью глубоких свёрточных сетей // Достижения в системах обработки нейронной информации (NeurIPS 2012). — 2012. — С. 1097–1105.
3. Симонян К., Циссерман Э. Очень глубокие свёрточные сети для крупномасштабного распознавания изображений // Архив научных публикаций. — 2014. — арХив:1409.1556. — Режим доступа: https://arxiv.org/abs/1409.1556 (дата обращения: 15.06.2026).
4. Хе К., Чжан С., Рен С., Сан Дж. Глубокие остаточные сети для распознавания изображений // Труды Конференции по компьютерному зрению и распознаванию образов (CVPR 2016). — 2016. — С. 770–778.

5. Тан М., Ле К. В. Эффективная масштабируемость сетей для компьютерного зрения // Труды Международной конференции по машинному обучению (ICML 2019). — 2019. — С. 6105–6114.
6. Лю З., Мао Х., Ву Ч. и др. Свёрточные сети на чистом пути: новое поколение архитектур для компьютерного зрения // Труды Конференции по компьютерному зрению и распознаванию образов (CVPR 2022). — 2022. — С. 11976–11986.
7. Гудфеллоу И., Бенджио И., Курвиль А. Глубокое обучение. — М.: ДМК Пресс, 2018. — 800 с.
8. Николенко С. И., Кадурин А., Архангельская Е. Глубокое обучение. — СПб.: Питер, 2018. — 416 с.
9. Документация по свёрточным слоям PyTorch [Электронный ресурс]. — Режим доступа: https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html (дата обращения: 15.06.2026).
10. Документация по свёрточным слоям TensorFlow Keras [Электронный ресурс]. — Режим доступа: https://www.tensorflow.org/api_docs/python/tf/keras/layers/Conv2D (дата обращения: 15.06.2026).

Чек-лист: что я должен уметь после изучения темы

После выполнения лабораторной работы студент должен уметь:

- ✓ представлять данные в виде тензоров и объяснять размерности N, C, H, W;
- ✓ различать математическую свёртку и кросс-корреляцию;
- ✓ рассчитывать размеры выходных тензоров для свёрточных слоёв;
- ✓ учитывать влияние `stride, padding и dilation`;
- ✓ вычислять число параметров свёрточного слоя;
- ✓ оценивать рецептивное поле сети;
- ✓ реализовывать базовую кросс-корреляцию вручную;
- ✓ проверять корректность реализации сравнением с библиотечной;
- ✓ строить и обучать свёрточные сети в PyTorch;
- ✓ анализировать фильтры и карты признаков;
- ✓ интерпретировать результаты обучения с точки зрения прикладного математика;
- ✓ понимать, когда свёрточные архитектуры уместны для анализа тензорных данных.